

TEMA 3. Excepciones

1. Empecemos un tema sobre control de errores en lenguajes de programación, con algo básico. En C, donde no existen las excepciones, pongamos un ejemplo de una raíz que toma número flotante positivo. Queremos controlar el error si la función recibe un número negativo. El usuario debe ser informado pero desde fuera de la función `raiz` ¿Cómo indicamos ese error?. Enumera dos opciones diferentes de diseñar, poniendo un ejemplo de código de cada una.

Opción 1: Retorno de un "Valor Centinela" (NaN) //Devuelve valor especial

Esta técnica consiste en devolver un valor especial que no sea un número real válido para indicar que algo salió mal. En matemáticas, la raíz de un número negativo no es un número real, por lo que devolvemos NaN (Not a Number).

Ventaja: No necesitas variables adicionales en la firma de la función.

Desventaja: El usuario debe acordarse de verificar el resultado con `isnan()`, o el error se propagará silenciosamente en los cálculos siguientes.

```

#include <stdio.h>
#include <math.h>

float raiz(float num) {
    if (num < 0) {
        return -1.0;
    }
    return sqrt(num);
}

int main() {
    float num = -5.0;
    float resultado = raiz(num);

    if (isnan(resultado)) {
        printf("Error: No se puede calcular la raiz de un numero negativo.\n");
    } else {
        printf("El resultado es: %.2f\n", resultado);
    }
    return 0;
}

```

Opción 2: Código de Estado y Puntero de Salida // Parametro adicional para almacenar un código de error

Este es el estándar en sistemas donde el rendimiento y la claridad son críticos. La función devuelve un int (0 para éxito, otro número para error) y el resultado real se "escribe" en una dirección de memoria proporcionada por el usuario a través de un puntero.

Ventaja: Separa claramente el estado de la ejecución del valor obtenido. E

Desventaja: La sintaxis es un poco más verbosa debido al uso de punteros.

```

#include <stdio.h>
#include <math.h>

// Retorna 0 si todo ok, -1 si hay error
int raiz_segura(float n, int *error) {
    if (n < 0) {
        *error=1;
        return 0; // Código de error
    }
    *error = sqrt(n);
    return sqrt (n); // Éxito
}

int main() {
    int error =0;
    float num = -10.0;
    float res = raiz (num, &error);

    if (error!=0) {
        printf("Error critico: Entrada invalida (%.2f).\n", num);
    } else {
        printf("La raiz es: %.2f\n", error);
    }
    return 0;
}

```

2. Brevemente ¿Qué es una "excepción"? ¿Con qué objetivo las usa un programador cuando implementa funciones o cuando las llama?

Una excepción es un evento inesperado o una condición de error que ocurre durante la ejecución de un programa y que interrumpe su flujo normal de instrucciones.

A diferencia de los códigos de error en C (donde el error viaja "en la mano" del valor de retorno), una excepción es como una alarma inteligente que salta por encima de las funciones hasta encontrar a alguien que sepa cómo apagarla.

¿Con qué objetivo las usa un programador?

Cuando un programador implementa o llama a una función, utiliza excepciones con tres metas

principales:

1.Separación de preocupaciones (Clean Code):

Permite separar el código que hace el trabajo sucio (la lógica de negocio) del código que maneja los desastres. Así, no llenas tu algoritmo principal de if (error) { ... } en cada línea.

2.Propagación Automática:

Si una función llamada a diez niveles de profundidad falla, la excepción "burbujea" hacia arriba automáticamente. No tienes que pasar el error manualmente de función en función a través de toda la jerarquía de llamadas.

3.Obligación de manejo (Robustez):

En muchos lenguajes, si una función puede lanzar una excepción crítica, el programador que la usa está obligado a decidir qué hacer con ella (capturarla o dejarla pasar). Esto evita que los errores pasen desapercibidos y el programa termine en un estado corrupto o "congelado".

En resumen: El objetivo es crear una red de seguridad que capture fallos (como un archivo que no existe o una división por cero) antes de que el programa colapse por completo, permitiendo que la aplicación se recupere o se cierre de forma elegante.

***CLASE-----

Excepción->

Surge en situaciones atípicas

Cuando implementamos -> Nos permite indicar más claramente el error

Cuando llamamos-> Me facilita separar la lógica normal de la de reacción o manejo de la situación errónea

3. Reescribe el mismo ejemplo de raíz, pero en Java, metiendo ese método en una clase Calculadora y

**llama a dicho método desde el método main ,
mostrando cómo se puede controlar desde fuera.**

```
public class Calculadora {

    /**
     * Calcula la raíz cuadrada de un número.
     * @param n El número flotante.
     * @return La raíz cuadrada.
     * @throws IllegalArgumentException si el número es negativo.
     */
    public static double raiz(double num) {
        if (num < 0.0) {
            // "Lanzamos" el objeto excepción. La función se detiene aquí.
            throw new IllegalArgumentException("No se puede calcular la raíz de un número negat:");
        }
        return Math.sqrt(num);
    }

    public static void main(String[] args) {
        double numero = -9.0;

        try {
            // Intentamos ejecutar el código que podría fallar
            System.out.println("Calculando...");
            double resultado = Calculadora.raiz(numero);

            // Esta línea NO se ejecutará si la línea anterior lanza una excepción
            System.out.println("El resultado es: " + resultado);

        } catch (IllegalArgumentException e) {
            // Capturamos el error "desde fuera" de la función
            System.out.println("ERROR capturado en main: " + e.getMessage());
        } finally {
            // Este bloque es opcional y se ejecuta siempre, haya error o no
            System.out.println("Finalizando operación de cálculo.");
        }
    }
}
```

4. ¿Qué es "lanzar" una excepción? ¿Qué es "controlar" o "capturar" una excepción? ¿Qué es que se "propague" una excepción? ¿Qué le va ocurriendo a las funciones en la pila de llamadas por donde se va propagando la excepción? ¿Las funciones que no la controlan se reanudan después de alguna forma? Explica con el mismo ejemplo anterior en Java de la raíz cuadrada.

1. ¿Qué es "lanzar" (throw) una excepción?

Es el acto de notificar que algo ha salido mal y que la función ya no puede cumplir su promesa. En nuestro ejemplo de Java, cuando `raiz(-9.0)` detecta el número negativo, "suelta" el testigo y hace sonar una alarma.

En el código: `throw new IllegalArgumentException(...)`.

Resultado: La función `raiz` se detiene inmediatamente. No llega a ejecutar el `return`.

2. ¿Qué es "propagar" una excepción?

Si la función que lanzó el error no sabe cómo arreglarlo, la excepción viaja hacia atrás por la pila de llamadas (call stack) buscando a alguien que sí sepa.

En el ejemplo: `raiz` le pasa el "problema" a `main`. Si `main` no tuviera un `try-catch`, se lo pasaría a la Máquina Virtual de Java (JVM), que finalmente detendría el programa.

3. ¿Qué le ocurre a las funciones en la pila de llamadas?

Aquí está la clave: Las funciones que no controlan la excepción se interrumpen bruscamente. Cuando la excepción se propaga por una función, esta "muere" (se saca de la pila) en ese mismo instante. Todas sus variables locales se destruyen y su ejecución se corta. No hay marcha atrás.

4. ¿Se reanudan las funciones que no la controlan?

No. Una vez que una excepción atraviesa una función sin ser capturada, esa función ha terminado para siempre.

Si `raiz` lanza la excepción, el código que estaba justo debajo de la llamada a `raiz` en el `main` nunca se ejecutará.

El control del programa salta directamente al bloque `catch`. Solo después de que el `catch` (o el `finally`) termine, el programa sigue su curso normal desde ese punto hacia adelante, pero nunca regresa a donde se quedó la función que falló.

```

public void calcularYMostrar() {
    double r = calc.raiz(-5); // 1. LANZA la excepción aquí.
    System.out.println(r);    // 2. ESTO NUNCA SE EJECUTA.
}

public static void main(String[] args) {
    try {
        instancia.calcularYMostrar(); // 3. La excepción SE PROPAGA hasta aquí.
    } catch (Exception e) {
        // 4. Aquí se CONTROLA / CAPTURA.
        System.out.println("Error manejado.");
    }
    // 5. El programa SIGUE desde aquí, pero calcularYMostrar() ya desapareció.
}

```

5. ¿Qué ventajas tiene frente a C, la "propagación natural" de las excepciones a través de la pila (*stack*) de llamadas?

La "propagación natural" es, posiblemente, el salto evolutivo más importante entre el manejo de errores "manual" de C y el manejo "estructurado" de Java o C++.

Aquí te detallo las ventajas estratégicas que ofrece frente al modelo de C:

1. Eliminación del "Efecto Burbujeo" Manual

En C, si una función a 10 niveles de profundidad encuentra un error, cada una de las 9 funciones intermedias debe

Recibir el código de error, Comprobarlo con un if, Retornarlo a la función superior.

Si un solo programador olvida un if en la cadena, el error se "silencia" y el programa sigue operando con datos corruptos. En Java, las funciones intermedias no tienen que hacer nada; la excepción las atraviesa automáticamente hasta encontrar un catch.

2. Reducción del "Código Espaguete" (Ruido Visual)

En C, el flujo lógico se mezcla constantemente con el control de errores.

3. Contexto Rico de Información (Stack Trace)

Cuando una excepción se propaga en Java, lleva consigo un objeto que contiene el Stack Trace (la traza de la pila).

En C, si recibes un -1, a veces es difícil saber qué función exacta de toda la cadena generó el error. En Java, la excepción sabe exactamente en qué archivo, qué método y en qué línea de código se originó el problema, y qué camino siguió hasta llegar a ti.

4. Gestión Centralizada de Errores

La propagación natural permite que decidas dónde es más inteligente manejar el error.

No tienes que manejar el error de la raíz dentro de una función matemática intermedia que no sabe qué decirle al usuario.

Puedes dejar que el error suba hasta la Capa de Interfaz de Usuario, que es el único lugar donde realmente se puede mostrar un mensaje de alerta con sentido o pedir nuevos datos.

5. Seguridad: "Falla rápido o no falles"

En C, es fácil ignorar un valor de retorno por accidente.

En lenguajes con excepciones, si no capturas un error crítico, el programa se detiene (crash).

Aunque parezca malo, es preferible que un programa se detenga a que siga funcionando con una raíz cuadrada negativa que luego corrompa una base de datos o el sistema de navegación de un avión.

***CLASE

El error se progaga de forma natural hasta que llega a una capa/funcion que sí sepa que hacer

En el main se capturan esas excepciones

6. En orientación a objetos, ¿las excepciones suelen ser objetos? ¿Qué ventajas tiene esto en términos de encapsulación? ¿Podemos entonces crear excepciones personalizadas?

1. ¿las excepciones suelen ser objetos?

Si, en los lenguajes modernos de Orientación a Objetos (OO) como Java, C#, C++ o Python, las excepciones son objetos

Esto significa que cuando ocurre un error, el sistema crea una instancia de una clase de excepción (como `ArithmeticException` o `FileNotFoundException`) que empaqueta todo lo que ocurrió en ese momento.

2. Ventajas en términos de Encapsulación

La encapsulación consiste en agrupar datos y comportamiento, ocultando la complejidad. Al ser objetos, las excepciones ofrecen ventajas:

-Estado Interno: Una excepción objeto puede guardar mucho más que un "mensaje de error". Puede almacenar la hora del fallo, el ID del usuario que lo provocó, el valor ilegal que causó el problema o incluso una referencia a la conexión de base de datos que falló.

-Comportamiento (Métodos): Las excepciones pueden tener métodos. Por ejemplo, una excepción de red podría tener un método `.getRetryDelay()` que le diga al programa cuánto tiempo esperar antes de reintentar, o `.logError()` para guardarse a sí misma en un archivo de diagnóstico.

-Jerarquía y Clasificación: Gracias a la herencia, podemos capturar errores de forma genérica o específica. Puedes capturar una `IOException` para manejar cualquier error de entrada/salida, o ser específico y capturar solo `FileNotFoundException`

3. ¿Podemos entonces crear excepciones personalizadas?

Sí, y es una de las mejores prácticas en el desarrollo profesional. Crear tus propias excepciones permite que tu código hable el "lenguaje del negocio".

En lugar de usar un genérico `IllegalArgumentException` para todo, se puede crear algo específico

7. En relación con las ventajas de la encapsulación, comparando el ejemplo en C con Java. ¿Qué información esencial lleva cualquier objeto excepción que es muy útil tener cuando se llega a un manejador?

Cuando una función en Java lanza una excepción y esta llega a manos del programador (en el catch), no recibe solo un aviso de "algo falló". Recibe un objeto que contiene tres datos vitales que en C no existen de forma automática:

1.El Nombre del Error (Identidad): En C, un -1 puede significar muchas cosas. En Java, el objeto tiene un nombre claro, como `RaizNegativaException`. Esto permite saber exactamente qué tipo de problema ocurrió sin tener que adivinar.

2.El Mensaje (La Explicación) (`getMessage`) : El objeto guarda una frase escrita por el programador que explica el fallo (ej: "Error: Intentaste calcular la raíz de -5"). Es como una etiqueta que te dice qué pasó exactamente en lenguaje humano.

3.La "Ruta del Crimen" (Stack Trace): Esta es la parte más útil. El objeto sabe exactamente en qué archivo, en qué función y en qué línea de código saltó el error. Además, dice qué funciones llamaron a qué funciones hasta llegar ahí.

4.Datos Extra (Contexto): Al ser un objeto, puedes guardar dentro cualquier cosa que ayude. Si falló una transferencia bancaria, el objeto puede llevar dentro el número de cuenta y el monto que falló. En C, tendrías que buscar esos datos por otro lado; en Java, vienen "dentro" del error.

***CLASE-----

-Un mensaje (getMessage())

-La traza de la pila (printStackTrace)-> vale para depurar

-Opcionalmente, la "Causa"-> es otra excepción que es la verdadera causa

8. En Java, sobre el bloque "try-catch", ¿se pueden tener más de un bloque catch ? ¿cuántos bloques catch se ejecutan?

1.¿Se pueden tener más de un bloque catch?

Sí. Un solo bloque try puede ir seguido de varios bloques catch diseñados para atrapar diferentes tipos de problemas.

Esto permite que el programa reaccione de forma específica según lo que haya fallado. No es lo mismo que falte un archivo a que el servidor de internet se haya caído.

2.¿Cuántos bloques catch se ejecutan?

Solo se ejecuta uno (o ninguno). Aunque tengas una lista de diez bloques catch, en el momento en que ocurre una excepción, Java busca de arriba hacia abajo cuál es el primero que "encaja" con el error.

Si encuentra uno que encaja, ejecuta ese bloque y se salta todos los demás.

Si no ocurre ningún error, no se ejecuta ninguno.

Ejemplo:

```

try {
    // 1. Código que puede fallar de varias formas
    descargarArchivo();
    double r = calcularRaiz(-5);
}
catch (FileNotFoundException e) {
    // Se ejecuta solo si el archivo no existe
    System.out.println("Error: No encontré el archivo.");
}
catch (RaizNegativaException e) {
    // Se ejecuta solo si el cálculo matemático falla
    System.out.println("Error: No puedo calcular esa raíz.");
}
catch (Exception e) {
    // Este es un "comodín" para cualquier otro error no previsto
    System.out.println("Ocurrió un error desconocido.");
}

```

IMPORTANTE: Un bloque try puede estar escoltado por múltiples bloques catch para gestionar distintos errores de forma personalizada. Sin embargo, ante un fallo, solo se ejecutará el primer catch que sea compatible con la excepción lanzada. Por ello, es obligatorio ordenar los bloques de lo más específico a lo más general.

***CLASE

Una AccessDenniedEception es un subtipo de IOException

1º el mas específico -> AccesDeniedException

2º el menos específico-> IOException

Si no se pone así los catch para excepciones específicas no se ejecutarán

9. Si las excepciones producen rupturas en el código llamador, ¿cómo podemos garantizar que se ejecuta siempre finalmente un código necesario para cierre de ficheros, liberación de recursos, antes de que continúe propagándose la excepción? Pon un

ejemplo en Java con finally , tanto con catch como sin él.

El Bloque finally: La Red de Seguridad

El bloque finally es una sección de código que se garantiza que se ejecutará siempre, independientemente de si se lanzó una excepción o si el código funcionó perfectamente. Es el lugar ideal para colocar las tareas de "limpieza" o liberación de recursos (cerrar un archivo, cerrar una conexión a Internet o una base de datos).

//El finally se ejecuta SIEMPRE que se ejecute en el bloque try

Reglas clave:

Ejecución garantizada: Se ejecuta si el try termina con éxito.

Ejecución tras el error: Se ejecuta después de que un catch maneje el error.

Ejecución en propagación: Si nadie captura el error, el bloque finally se ejecuta justo antes de que la excepción siga subiendo por la pila de llamadas hacia la siguiente función.

Ejemplo 1:

Con catch (Manejo completo)

Se usa cuando quieres arreglar el error y además limpiar recursos.

```
try {
    abrirArchivo();
    System.out.println("Leyendo dato: " + calc.raiz(-5));
} catch (RaizNegativaException e) {
    System.out.println("Error matemático detectado.");
} finally {
    // Esto se ejecuta SIEMPRE: tanto si la raíz fue bien como si saltó el catch
    cerrarArchivo();
    System.out.println("Recurso liberado.");
}
```

Ejemplo 2:

Sin catch (Solo limpieza)

A veces no quieres capturar el error (prefieres que se propague a una función superior), pero sí tienes la obligación de cerrar lo que abriste.

```

public void procesar() throws Exception {
    try {
        abrirConexionInternet();
        hacerCalculoComplicado(); // Si esto falla, la función se interrumpe
    } finally {
        // Aunque el error suba a la función de arriba,
        // Java no se olvida de pasar por aquí antes de irse.
        cerrarConexionInternet();
        System.out.println("Conexión cerrada antes de propagar el error.");
    }
}

```

Resumen: El bloque finally actúa como un seguro de cierre. Su propósito principal es evitar la fuga de recursos.

Mientras que el catch es para decidir qué hacer con el problema, el finally es para dejar la casa limpia antes de continuar o de que el programa se detenga por el error.

10. En Java, el bloque finally puede ir sin catch ? ¿Se ejecuta siempre tanto si ocurre como si no ocurre una excepción? ¿Y si hay un return en medio del try ?

1.Sí. La estructura mínima requerida es try-finally. Esto es útil cuando no quieres (o no sabes cómo) manejar la excepción en ese punto, pero necesitas asegurarte de que ciertos recursos se cierren o se limpien (como cerrar un archivo o una conexión a base de datos) antes de que la excepción se propague hacia arriba.

```

try {
    // Código que podría lanzar una excepción
} finally {
    // Código que se ejecuta sí o sí
}

```

2.¿Se ejecuta siempre, ocurra o no una excepción?

Si. El bloque finally está diseñado para ejecutarse independientemente de si el flujo fue exitoso o si saltó al catch.

Sin embargo, existen excepciones extremas donde no se ejecutaría:

Si el hilo muere o la Máquina Virtual (JVM) se detiene abruptamente.

Si invocas explícitamente a `System.exit(0)`.

Si ocurre un fallo catastrófico de hardware o un error fatal del sistema operativo.

3. ¿Y si hay un `return` en medio del `try`?

Esta es la parte más curiosa: El bloque `finally` gana.

Si tienes un `return` dentro del bloque `try`, Java primero evalúa la expresión del `return`, pero antes de entregar el control a quien llamó al método, ejecuta el contenido del `finally`.

```
public int prueba() {  
    try {  
        return 1;  
    } finally {  
        System.out.println("Ejecutando finally...");  
    }  
}
```

En este código primero se verá el mensaje "Ejecutando finally..." en consola y luego el método devolverá el valor 1.

11. En Java, qué son las excepciones "**controladas**" y las "**no controladas**"? ¿Qué papel juega

RuntimeException ? Pon un ejemplo de excepciones típicas controladas y no controladas que incluso nosotros mismos podríamos usar. Haz dos listas con 3 o 4 ejemplos de situación donde se suele preferir una excepción controlada y donde se suele preferir una excepción no controlada.

1. Excepciones Controladas (Checked)

Son aquellas que el compilador te obliga a gestionar. Si un método lanza una excepción controlada, tienes dos opciones: envolverla en un bloque `try-catch` o declararla en la firma del método usando `throws`.

Herencia: Heredan de `Exception` (pero no de `RuntimeException`).

Filosofía: Se usan para condiciones de error que son recuperables y que no dependen directamente del programador, sino de factores externos (red, archivos, base de datos).

2.Excepciones No Controladas (Unchecked)

Son aquellas que el compilador ignora. No estás obligado a capturarlas ni a declararlas.

Herencia: Heredan de RuntimeException o de Error.

Filosofía: Se usan para errores de lógica o mal uso de la API (errores de programación). Se asume que el programador debería haber evitado que esto ocurriera en primer lugar.

3.El papel de RuntimeException

RuntimeException es la "madre" de todas las excepciones no controladas. Su papel es actuar como una línea divisoria:

Si tu excepción hereda de ella, el compilador te deja libre.

Si hereda de Exception a secas, el compilador se pone estricto.

Ejemplos típicos:

-Controlada: IOException

-No Controlada: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException (como dividir por cero).

***CLASE

Tipos de RuntimeException: -> NO CONTROLADAS -> tu no estas obligado a colocar a bloques try catch /throws

-IllegalArgument

-NullPointerException : cuando uno intenta llamar un método sobre una referencia que está apuntando a null

-ArrayIndexOutOfBoundsException

Tipos de IOException -> CONTROLADAS -> Si obliga a try catch/throws

-AccessDeniedException

4.Ejemplos de cuando se prefiere una u otra

Preferimos Excepciones Controladas (Checked)

Se usan cuando el error es una contingencia razonable del mundo real.

Fallo en servicios externos: Cuando intentas conectar con una API de pagos y el servidor no responde.

Problemas de Archivos: Intentar leer un archivo de configuración que el usuario pudo haber borrado por error.

Restricciones de Negocio: Por ejemplo, en una app bancaria, lanzar un `SaldoInsuficienteException` para obligar al programador a decidir qué mostrar al usuario si no hay dinero.

Base de datos: Cuando una consulta falla porque la tabla está bloqueada temporalmente.

Preferimos Excepciones No Controladas (Unchecked)

Se usan cuando el error indica que el programador "metió la pata" o los datos son inválidos.

Argumentos Inválidos: Si un método recibe un número negativo donde solo se esperan positivos (`IllegalArgumentException`).

Estado Ilegal: Intentar disparar un arma en un juego cuando no tiene munición (`IllegalStateException`).

Punteros Nulos: Cuando intentas acceder a un objeto que no ha sido inicializado (`NullPointerException`).

Errores de Casting: Intentar convertir un objeto Gato en un objeto Perro (`ClassCastException`).

TRUCO: Si el cliente del código no puede hacer nada para solucionar el problema de forma automática, usa una No Controlada. Si el cliente debe tomar una decisión para recuperarse, usa una Controlada.

12. ¿Qué es y para qué se usa `throws` ? ¿Por qué es alternativa a capturar una excepción controlada?

1. ¿Qué es y para qué se usa?

En Java, `throws` es una palabra clave que se utiliza en la declaración de un método para indicar que dicho método puede lanzar una o más excepciones durante su ejecución.

Es, en esencia, un cartel de advertencia para quien sea que llame a ese método.

Cuando usas `throws`, estás delegando la responsabilidad de manejar el error. En lugar de detener el flujo y resolver el problema ahí mismo, el método dice: "Yo no voy a lidiar con esto; que se encargue el que me invocó".

Sintaxis: Se coloca después de los paréntesis de los argumentos y antes de la llave de apertura.

Uso principal: Se utiliza casi exclusivamente con excepciones controladas (`IOException`), ya que el compilador exige que estas se declaren o se capturen.

2. ¿Por qué es una alternativa a capturar la excepción?

Normalmente, ante una excepción controlada, tienes dos caminos (la famosa regla del "Catch or Specify"):

Catch (Capturar): Usas try-catch. Resuelves el problema en el sitio.

Specify (Especificar con throws): No pones try-catch, pero añades throws a la firma.

13. Pon un ejemplo en Java de firma de método que incluya throws , de una función que abre un fichero pero que declara que no le interesa manejar la excepción de si el fichero no existe, sino que se propague hacia arriba. Eso sí, acuérdate del finally .

```
public String leerFichero (Path p) throws IOException{
    try{
        ...=Files.readAllBytes(p);

    }finally{

    }
}
```

14. ¿Podemos poner en throws excepciones no controladas, como RuntimeException ? ¿Debería el método llamador entonces poner try-catch en ese caso? ¿Qué sentido tendría?

Si, por poder podemos, pero el compilador no va a obligar a poner un bloque try..catch (no es lo habitual)

1. ¿Se puede poner una RuntimeException en el throws?

Sí. Java te permite declarar cualquier excepción en la firma del método, ya sea checked (como IOException) o unchecked (como NullPointerException o ArithmeticException). El compilador no te obliga, pero no te lo prohíbe.

2. ¿Debe el método llamador usar try-catch?

No es obligatorio. A diferencia de las excepciones controladas, el compilador de Java no "protestará" si el método que llama ignora una `RuntimeException`.

Si no pones try-catch, el programa compila perfectamente.

Si la excepción ocurre en tiempo de ejecución y nadie la atrapa, el programa simplemente se detendrá (o el hilo morirá).

3. ¿Qué sentido tiene hacerlo?

Si no es obligatorio, ¿para qué ensuciar la firma del método? Principalmente por dos razones:

Documentación (Contrato del método): Es una forma de decirle a otros programadores: "Oye, aunque no te obligue el compilador, ten cuidado porque este método podría lanzar un `IllegalArgumentException` si me pasas un dato absurdo". Ayuda a que quien use tu código sepa a qué atenerse.

Claridad en APIs grandes: En frameworks o librerías complejas, declarar una `RuntimeException` específica ayuda a que las herramientas de generación de documentación (como Javadoc) incluyan esa advertencia de forma explícita.

15. ¿Cuándo se recomienda usar excepciones controladas, como `IOException`, y cuándo no controladas como `IllegalArgumentException`? ¿Existen en todos los lenguajes ambas opciones? En los que sólo existe una opción, ¿cuál es la más habitual?

La distinción principal se basa en si el que llama al método puede hacer algo para arreglar el problema o no.

Excepciones Controladas (`IOException`):

Se usan para condiciones de las que el programa puede recuperarse razonablemente.

Ejemplo: Un archivo no encontrado o un fallo de red. No es culpa del programador que el Wi-Fi se caiga. El programa debe estar obligado a manejarlo (ej. reintentar o pedir otra ruta).

Excepciones No Controladas (`RuntimeException`):

Se usan para errores de programación.

Ejemplo: Pasar un null donde no debe haberlo (NullPointerException) o un índice fuera de rango (ArrayIndexOutOfBoundsException). Estos son errores que el programador debería haber evitado con lógica previa, no con un try-catch.

2. ¿Existen ambas en todos los lenguajes?

No, Java es prácticamente el único language que implementa y defiende las excepciones controladas de forma estricta.

C++, C#, Python, Ruby, JavaScript: Todos estos lenguajes solo tienen excepciones no controladas. Puedes lanzar cualquier excepción, pero el compilador nunca te obligará a capturarla.

Go y Rust: Estos lenguajes ni siquiera usan "excepciones" en el sentido tradicional para errores recuperables. Devuelven el error como un valor más de la función (en Rust se usa el tipo Result).

3. ¿Cuál es la opción más habitual?

La tendencia moderna en la ingeniería de software es eliminar las excepciones controladas. La opción más habitual es que solo existan las excepciones no controladas.

¿Por qué la industria se aleja de las "Checked Exceptions"?

Aunque la idea de Java era buena (obligar a la robustez), en la práctica generó dos problemas:

Código sucio: Muchos programadores, por pereza, acaban poniendo bloques catch vacíos solo para que el código compile, lo que silencia errores graves.

Rigidez: Si cambias una excepción en lo profundo de una librería, tienes que actualizar las firmas de cientos de métodos hacia arriba en la cadena de llamadas.

16. ¿Tiene sentido lanzar excepciones dentro del catch ? ¿Se puede relanzar la misma excepción capturada? ¿Cuándo tendría sentido hacer esto último? Pon ejemplos de ambos casos.

Si, tiene sentido

-Relanzar la misma excepcion ->

```
try{

}catch (NumberFormatException e ){
    throw e;
}
```

-Envolver en otra excepción nueva ->

```
try{                                                    //Captamos una excepcion

}catch (IOException e){
    throw new RuntimeException ("excepcion de E/S", E);    //Lanzamos una nueva
}
```

-Lanzar otra excepcion totalmente nueva ->

```
try{

}catch (IOException e){
    throw new AplicacionExeption ("error");
}
```

17. ¿En qué consiste que una excepción sea la "causa" de otra excepción? Pon un ejemplo en Java, donde capturemos una excepción de bajo nivel y la encapsulemos en otra personalizada de alto nivel. Cuando una excepción sale por pantalla y tiene una causa, ¿se ve?

Consiste en que yo capturo la exception y lanzo la excepcion original como causa primigenia

```

try{                                                    //Captamos una excepcion

}catch (IOException e){

    throw new RuntimeException ("excepcion de E/S", E);    //Lanzamos una nueva
}

```

Una excepcion como causa de la otra

-Se ve cuando la excepcion se muestra por pantalla

"Excepcion externa (NetfluxException)

.-----

.-----

.-----

"Caused by excepcion interna (IOException)

.-----

.-----

.-----

-Se puede obtener con el método "getCause()"